

Módulo 2 – Modelagem, Modelagem, Documentação e Execução

Nome: Mariana Caetano Costa

Email: mariana.cae.costa@gmail.com

Telefone: (31) 9.8859-0581

1. Diferença entre Checking e Testing

Checking é verificar se algo está de acordo com um resultado ou requisito previamente definido. É uma atividade mais objetiva, baseada em confirmar se o sistema está funcionando conforme esperado.

Testing é uma atividade investigativa, que busca entender o comportamento do software, explorar possibilidades, identificar riscos e encontrar problemas que podem não estar previstos nos requisitos.

Em resumo: Checking verifica; Testing investiga.

2. Testes AD HOC

São testes realizados **sem um planejamento ou roteiro formal**, utilizando a experiência e a percepção do testador para explorar o sistema.

O objetivo é encontrar problemas de forma rápida, explorando funcionalidades, combinações ou situações que podem não estar contempladas nos casos de teste.

Exemplo: acessar uma tela e experimentar diferentes combinações de campos, valores e ações sem seguir um caso de teste previamente definido.

3. Testando com Heurísticas de Teste

Heurísticas são **regras práticas ou estratégias** utilizadas para orientar a investigação durante os testes.

Elas ajudam o testador a pensar em diferentes possibilidades e áreas de risco, principalmente quando não existe informação suficiente para criar testes completos.

Exemplo: verificar campos com valores vazios, valores muito grandes, caracteres especiais, formatos inválidos e combinações inesperadas.

4. Teste Exploratório Estruturado — Introduzindo o Gerenciamento de Testes Baseado em Sessão

O **Teste Exploratório** combina aprendizado, criação e execução dos testes ao mesmo tempo.

No **Gerenciamento de Testes Baseado em Sessão (SBTM — Session-Based Test Management)**, o trabalho exploratório é organizado em sessões com um objetivo definido.

Uma sessão normalmente possui:

- objetivo da sessão;
- área ou funcionalidade a ser explorada;
- tempo definido;
- testes realizados;
- problemas encontrados;
- observações e aprendizados.

Isso permite manter a liberdade do teste exploratório, mas com maior organização e rastreabilidade.

5. Teste Exploratório Estruturado — Exemplo Prático

Na prática, uma sessão de teste exploratório pode ser criada para investigar uma funcionalidade específica.

Exemplo:

Objetivo: explorar o fluxo de login.

Durante a sessão, o testador pode investigar:

- login válido;
- senha incorreta;
- usuário inexistente;
- campos vazios;
- caracteres especiais;
- tentativas consecutivas;
- comportamento após bloqueio;
- mensagens de erro.

Ao final, são registrados os testes realizados, problemas encontrados, dúvidas e possíveis melhorias.

6. Aplicando a Partição de Equivalência

A **Partição de Equivalência** divide os valores possíveis de entrada em grupos ou classes que devem apresentar comportamentos semelhantes.

Assim, não é necessário testar todos os valores possíveis.

Exemplo: um campo aceita idade de 18 a 60 anos.

Podemos criar:

- Classe inválida: menor que 18;
- Classe válida: 18 a 60;
- Classe inválida: maior que 60.

Em vez de testar todas as idades, selecionamos representantes de cada classe.

7. Aplicando a Análise do Valor Limite

A **Análise do Valor Limite** concentra os testes nos valores próximos aos limites permitidos, pois erros costumam ocorrer nessas regiões.

Para um campo que aceita valores de **18 a 60**, podemos testar:

- 17 → abaixo do limite;
- 18 → limite inferior;
- 19 → próximo ao limite;
- 59 → próximo ao limite;
- 60 → limite superior;
- 61 → acima do limite.

Essa técnica complementa a Partição de Equivalência.

Por exemplo:

```
Console.WriteLine("Digite a idade: ");
int idade = int.Parse(Console.ReadLine());

if (idade >= 18 && idade <= 60){
    Console.WriteLine("Idade válida");
}
else{
    Console.WriteLine("Idade inválida");
}
```

8. Aplicando a Tabela de Decisão

A **Tabela de Decisão** é utilizada quando o comportamento do sistema depende da combinação de diferentes condições.

Ela apresenta:

- condições;
- possíveis valores dessas condições;
- ações esperadas.

Exemplo: para aprovar um aluno:

Média	Frequência	Resultado
≥ 7	≥ 75%	Aprovado

Média	Frequência	Resultado
≥ 4	$\geq 75\%$	Recuperação
< 4	$\geq 75\%$	Reprovado
qualquer	$< 75\%$	Reprovado

A técnica ajuda a garantir que diferentes combinações de condições sejam testadas.

```
double media = 7;
double frequencia = 80;

if (frequencia < 75){
    Console.WriteLine("Reprovado");
}
else if (media >= 7){
    Console.WriteLine("Aprovado");
}
else if (media >= 4){
    Console.WriteLine("Recuperação");
}
else{
    Console.WriteLine("Reprovado");
}
```

9. Aplicando Cobertura de Sentença e Cobertura de Decisão

Essas técnicas são utilizadas para avaliar quanto do código foi exercitado pelos testes.

Você senta junto com o Dev e discute qual o melhor testes a ser implementado. Teste de unidade é feito pelo desenvolvedor.

Cobertura de Sentença: verifica se as instruções do código foram executadas pelo menos uma vez.

Cobertura de Decisão: verifica se cada decisão foi testada em seus diferentes resultados, como **verdadeiro** e **falso**.

Por exemplo:

```
if (idade >= 18){
    Console.WriteLine("Maior de idade");
}
Else {
    Console.WriteLine("Menor de idade");
}
```

Para obter cobertura de decisão, precisamos testar pelo menos:

- idade >= 18 → verdadeiro;
- idade >= 18 → falso.

10. Documentando Condições de Teste

A **condição de teste** representa aquilo que será verificado no sistema.

É uma forma de transformar requisitos ou regras de negócio em pontos que precisam ser testados.

Documentos formais com templates pré-definidos para padronizar a documentação.

Exemplos:

- **Plano de Teste** → objetivo, escopo, estratégia, ambiente, riscos etc.
- **Caso de Teste** → ID, pré-condições, passos, dados, resultado esperado e resultado obtido.
- **Cenário de Teste** → funcionalidade e condição que será validada.
- **Script de Teste** → sequência detalhada de ações para execução.
- **Relatório de Defeito/Bug** → título, ambiente, passos para reproduzir, resultado esperado, resultado atual, evidências e severidade.
- **Checklist de Testes** → lista de verificações a serem executadas.

Teste: feito pelo ser humano envolve exploração, cognição, observação, inferência. Planeja em alto nível, define o que vai testar nos testes exploratórios.

Checagem: você tem os requisitos, extrai as regras de negócio e o que vai ser testado. Planejado antecipadamente.

Exemplo:

Requisito: O usuário deve conseguir realizar login utilizando usuário e senha válidos.

Condições de teste:

- usuário válido;
- senha válida;
- usuário inválido;
- senha inválida;
- campos vazios.

<pre>string usuario = "admin"; string senha = "123456"; if (usuario == "" senha == "") { Console.WriteLine("Campos vazios"); } else if (usuario == "admin" && senha == "123456") { Console.WriteLine("Login realizado"); } else if (usuario != "admin" && senha == "123456") { Console.WriteLine("Usuário inválido"); } else if (usuario == "admin" && senha != "123456") { Console.WriteLine("Senha inválida"); } else { Console.WriteLine("Usuário e senha inválidos"); }</pre>	Para testar todas as condições 1. admin / 123456 → Login realizado 2. usuario / 123456 → Usuário inválido 3. admin / 111111 → Senha inválida 4. usuario / 111111 → Usuário e senha inválidos 5. "" / 123456 → Campos vazios 6. admin / "" → Campos vazios 7. "" / "" → Campos vazios
---	--

11. Documentando Casos de Teste

O **caso de teste** detalha como uma determinada condição será verificada.

Exemplo:

Id	CT-001
Título	Login com usuário e senha válidos
Prioridade	Alta
Rastreabilidade	REQ-001
Pré-Condições	Usuário cadastrado e sistema disponível

Passos	Ação	Resultado Esperado
--------	------	--------------------

Passo 1	Informar usuário admin, informar senha 123456 e clicar em "Entrar".	Usuário deve acessar o sistema com sucesso.
Pós-Condições	Usuário autenticado no sistema	

12. Usando Gherkin para Documentar Testes

O **Gherkin** permite descrever comportamentos do sistema utilizando uma linguagem estruturada e de fácil entendimento.

Criado para ser utilizado dentro da ferramenta **Cucumber**, é uma ferramenta de suporte ao **BDD (Behavior-Driven Development – Desenvolvimento Orientado a Comportamento)**.

Importante: Cucumber é a ferramenta; **Gherkin é a linguagem** usada para escrever os cenários que o Cucumber interpreta.

Os principais elementos são:

- **Feature** → funcionalidade;
- **Scenario** → cenário;
- **Given** (dado que) → condição inicial/pré-condição.
- **When** (quando) → ação realizada
- **Then** (então) → resultado esperado
- **And** (e) → complementa uma condição ou ação.
- **@** → adiciona tags

Exemplo:

Feature: Login

@PrioridadeBaixa

Scenario: Login com dados válidos

Given que estou na tela de login

When informo usuário e senha válidos

And clico em "Entrar"

Then devo acessar o sistema

13. Criando Documentação de Testes no Jira e Zephyr

O **Jira** pode ser utilizado para organizar o trabalho e acompanhar as atividades de desenvolvimento e testes.

Com o **Zephyr** (plugin), é possível gerenciar artefatos de teste, como:

- casos de teste;
- ciclos de teste;
- execuções;
- resultados;
- evidências;
- defeitos.

A documentação fica centralizada e relacionada às demandas do projeto.

14. Executando Casos de Teste no Jira e Zephyr

Depois de criar os casos de teste, eles podem ser executados utilizando o Zephyr.

Durante a execução, o testador registra o resultado de cada caso, normalmente como:

- **Passed** → passou;
- **Failed** → falhou;
- **Blocked** → bloqueado;
- **Not Executed** → não executado.

Quando um teste falha, o resultado pode ser associado a um **defeito no Jira**, facilitando a rastreabilidade.

15. Reportando Defeitos no Jira

O registro de defeitos deve apresentar informações suficientes para que o desenvolvedor consiga **entender, reproduzir e corrigir o problema**.

Um bom defeito pode conter:

- Título
- Descrição
- Ambiente
- Pré-condição
- Passos para reprodução
- Resultado esperado
- Resultado obtido
- Evidencias
- Severidade
- Prioridade

Exemplo de título:

Login permite acesso com senha inválida

O objetivo é tornar o defeito **claro, reproduzível e rastreável**.